

Team Black — Limelight Vision Implementation

Version: 1.0 Post-Networking Setup Guide System: Limelight 3 + 2026 KitBot

This guide assumes networking is complete and the Limelight dashboard is accessible at <http://10.13.51.11:5801>. It covers pipeline configuration, vendor library installation, all new Java files, and the exact changes to RobotContainer.java — verified against the 2026 KitBot source code.

STEP 1: Configure the Limelight Pipeline

The pipeline tells the Limelight what to look for. This must be set before any code will detect tags.

In the Limelight Dashboard (<http://10.13.51.11:5801>)

- ☐ Go to the Pipelines tab.
- ☐ Confirm you are on pipeline index 0 — this is what the robot code reads by default.
- ☐ Set the pipeline type to: AprilTag
- ☐ Set Tag Family to: tag36h11
- ☐ Go to the Settings tab and set Stream Resolution to 320x240 and FPS to 15.
- ☐ Hold an AprilTag in front of the Limelight and confirm a green box appears around it in the dashboard feed.
- ☐ Confirm TX, TY, and TV values update on the right side of the dashboard when a tag is visible.

NOTE: TX = horizontal angle to tag (yaw). TY = vertical angle (pitch). TV = 1 if a target is visible, 0 if not. These are the three values the robot code uses.

STEP 2: Install the Limelight Vendor Library

The vendor library gives the robot code access to the LimelightHelpers class. Without it, the code will not compile.

In WPILib VS Code

- ☐ Press Ctrl+Shift+P to open the command palette.
- ☐ Type and select: WPILib: Manage Vendor Libraries
- ☐ Select: Install new libraries (online)
- ☐ Paste the following URL and press Enter:

<https://limelightvision.io/vendordeps/LimelightLib.json>

- ☐ If that URL fails, try the GitHub fallback:

<https://raw.githubusercontent.com/LimelightVision/limelightlib-wpijava/main/LimelightLib.json>

- ☐ When prompted, run a Gradle build to confirm the install succeeded.
- ☐ Open build.gradle and confirm a limelight entry appears in the dependencies block.

WARNING: Do not skip the build step. If the library is not properly installed, all LimelightHelpers calls will show as unresolved errors and the code will not deploy.

STEP 3: Create VisionSubsystem.java

This is a brand new file. Create it at exactly this path in your project:

`src/main/java/frc/robot/subsystems/VisionSubsystem.java`

Copy the entire file contents below exactly as written. Update the four measurement constants in the marked section after deploying (see Step 5).

Full File: VisionSubsystem.java

```
// Copyright (c) FIRST and other WPILib contributors.
// Open Source Software; you can modify and/or share it under the terms of
// the WPILib BSD license file in the root directory of this project.

package frc.robot.subsystems;

import edu.wpi.first.math.util.Units;
import edu.wpi.first.wpilibj2.command.SubsystemBase;
import frc.robot.LimelightHelpers;

public class VisionSubsystem extends SubsystemBase {

    // -----
    // MEASURE YOUR ROBOT AND UPDATE THESE VALUES (See Step 5)
    // -----
    private static final double CAMERA_HEIGHT_METERS =
        Units.inchesToMeters(20.0); // Floor to center of Limelight lens
    private static final double TARGET_HEIGHT_METERS =
        Units.inchesToMeters(51.25); // Floor to center of AprilTag on goal
        // SET THIS FROM THE 2026 GAME MANUAL
    private static final double CAMERA_PITCH_RADIANS =
        Units.degreesToRadians(15.0); // Upward tilt angle of Limelight
    private static final double DESIRED_RANGE_METERS = 2.0;
        // Shooting distance bumper to goal

    // -----
    // PID TUNING CONSTANTS (See Step 5)
    // -----
    private static final double kP_ROTATION = 0.04;
    private static final double kP_DISTANCE = 0.6;

    // Must match the name shown in the Limelight dashboard Settings tab
    private static final String LIMELIGHT_NAME = "limelight";

    /**
     * Returns [forwardSpeed, rotationSpeed] for auto-alignment.
     * Returns [0.0, 0.0] if no target is visible.
     * Forward speed drives to DESIRED_RANGE_METERS.
     * Rotation speed centers the tag horizontally in frame.
     */
    public double[] getAlignmentSpeeds() {
        // TV: 1.0 if a target is visible, 0.0 if not
        boolean hasTarget = LimelightHelpers.getTV(LIMELIGHT_NAME);

        if (hasTarget) {
            // TX: horizontal angle to target in degrees
            // Negative = target is left, positive = target is right
            double yaw = LimelightHelpers.getTX(LIMELIGHT_NAME);

            // TY: vertical angle to target in degrees
            // Used in trig-based distance calculation below
        }
    }
}
```

```

        double pitch = LimelightHelpers.getTY(LIMELIGHT_NAME);

        // Rotation: proportional to how far off-center the tag is
        double rotationSpeed = yaw * kP_ROTATION;

        // Distance via trigonometry using physical robot measurements
        // More accurate than Limelight's built-in pixel-size estimate
        double angleToTarget = CAMERA_PITCH_RADIANS
            + Units.degreesToRadians(pitch);
        double currentDist =
            (TARGET_HEIGHT_METERS - CAMERA_HEIGHT_METERS)
            / Math.tan(angleToTarget);

        // Forward speed: positive = drive forward, negative = back up
        double forwardSpeed =
            (currentDist - DESIRED_RANGE_METERS) * kP_DISTANCE;

        // Safety caps: limit max speed in either direction
        return new double[]{
            Math.max(-0.5, Math.min(0.5, forwardSpeed)), // index 0
            Math.max(-0.4, Math.min(0.4, rotationSpeed)) // index 1
        };
    }

    // No target visible - stop all movement
    return new double[]{0.0, 0.0};
}
}

```

STEP 4: Update RobotContainer.java

Three additions are made to RobotContainer.java. Every line of the original code is kept exactly as-is. Only new lines are added. The full file is shown below with the additions clearly marked in comments.

WARNING: Do NOT remove or change any existing code. Do NOT change operatorController bindings. Do NOT change the Drive default command. Only add the three marked sections below.

Full File: RobotContainer.java

```

// Copyright (c) FIRST and other WPILib contributors.
// Open Source Software; you can modify and/or share it under the terms of
// the WPILib BSD license file in the root directory of this project.

package frc.robot;

import edu.wpi.first.wpilibj.smartdashboard.SendableChooser;
import edu.wpi.first.wpilibj2.command.Command;
import edu.wpi.first.wpilibj2.command.RunCommand; // NEW
import edu.wpi.first.wpilibj2.command.button.CommandXboxController;
import edu.wpi.first.wpilibj2.command.button.Trigger;
import static frc.robot.Constants.OperatorConstants.*;
import frc.robot.commands.Drive;
import frc.robot.commands.Eject;

```

```

import frc.robot.commands.ExampleAuto;
import frc.robot.commands.Intake;
import frc.robot.commands.LaunchSequence;
import frc.robot.subsystems.CANDriveSubsystem;
import frc.robot.subsystems.CANFuelSubsystem;
import frc.robot.subsystems.VisionSubsystem;           // NEW

/**
 * This class is where the bulk of the robot should be declared. Since
 * Command-based is a declarative paradigm, very little robot logic should
 * actually be handled in the Robot periodic methods (other than the
 * scheduler calls). Instead, the structure of the robot (including subsystems,
 * commands, and trigger mappings) should be declared here.
 */
public class RobotContainer {
    // The robot's subsystems
    private final CANDriveSubsystem driveSubsystem = new CANDriveSubsystem();
    private final CANFuelSubsystem fuelSubsystem = new CANFuelSubsystem();
    private final VisionSubsystem visionSubsystem = new VisionSubsystem(); // NEW

    // The driver's controller
    private final CommandXboxController driverController =
        new CommandXboxController(DRIVER_CONTROLLER_PORT);

    // The operator's controller
    private final CommandXboxController operatorController =
        new CommandXboxController(OPERATOR_CONTROLLER_PORT);

    // The autonomous chooser
    private final SendableChooser<Command> autoChooser = new SendableChooser<>();

    /**
     * The container for the robot. Contains subsystems, OI devices, and
     * commands.
     */
    public RobotContainer() {
        configureBindings();

        // Set the options to show up in the Dashboard for selecting auto modes.
        // If you add additional auto modes you can add additional lines here
        // with autoChooser.addOption
        autoChooser.setDefaultOption("Autonomous",
            new ExampleAuto(driveSubsystem, fuelSubsystem));
    }

    /**
     * Use this method to define your trigger->command mappings.
     */
    private void configureBindings() {

        // While the left bumper on operator controller is held, intake Fuel
        operatorController.leftBumper().whileTrue(new Intake(fuelSubsystem));
        // While the right bumper on the operator controller is held, spin up
        // for 1 second, then launch fuel. When the button is released, stop.
        operatorController.rightBumper().whileTrue(new
LaunchSequence(fuelSubsystem));
        // While the A button is held on the operator controller, eject fuel
        operatorController.a().whileTrue(new Eject(fuelSubsystem));
    }
}

```

```

// Set the default command for the drive subsystem to the command
// provided by factory with the values provided by the joystick axes
// on the driver controller. The Y axis of the controller is inverted
// so that pushing the stick away from you (a negative value) drives
// the robot forwards (a positive value)
driveSubsystem.setDefaultCommand(
    new Drive(driveSubsystem, driverController));

fuelSubsystem.setDefaultCommand(
    fuelSubsystem.run(() -> fuelSubsystem.stop()));

// -----
// NEW: B button on driver controller activates vision align
// Holds B = robot rotates toward tag and drives to target
// Releases B = robot stops and returns to manual Drive command
// -----
driverController.b().whileTrue(new RunCommand(() -> {
    double[] speeds = visionSubsystem.getAlignmentSpeeds();
    // speeds[0] = forward/back, speeds[1] = rotation
    driveSubsystem.driveArcade(speeds[0], speeds[1]);
}, driveSubsystem));
}

/**
 * Use this to pass the autonomous command to the main Robot class.
 *
 * @return the command to run in autonomous
 */
public Command getAutonomousCommand() {
    // An example command will be run in autonomous
    return autoChooser.getSelected();
}
}

```

STEP 5: Deploy and Measure

Once both files are saved, deploy to the robot. Then update the measurement constants in VisionSubsystem.java with your real values and redeploy.

Deploy

- ☐ Press Ctrl+Shift+P and select: WPILib: Deploy Robot Code
- ☐ Confirm the build is SUCCESSFUL in the terminal output.
- ☐ If you see any LimelightHelpers import errors, the vendor library did not install correctly. Repeat Step 2.
- ☐ Open Shuffleboard and confirm limelight appears in the NetworkTables tree with tv, tx, ty values.

Physical Measurements to Update in VisionSubsystem.java

Constant	What to Measure	How to Update
CAMERA_HEIGHT_METERS	Floor to center of Limelight lens.	Change 20.0 to your height in inches inside Units.inchesToMeters()
TARGET_HEIGHT_METERS	Floor to center of AprilTag on goal.	Set to exact value from 2026 Game Manual inside Units.inchesToMeters()
CAMERA_PITCH_RADIANS	Upward tilt angle of Limelight from horizontal.	Change 15.0 to your angle in degrees inside Units.degreesToRadians()
DESIRED_RANGE_METERS	Bumper to goal face at your shooting sweet spot.	Change 2.0 to your measured distance in meters directly

WARNING: TARGET_HEIGHT_METERS must come from the 2026 Game Manual. Do not guess or estimate this value — it directly controls where the robot stops.

Verify Alignment Works

- ☐ Enable Teleop from the Driver Station.
- ☐ Point the robot roughly toward an AprilTag.
- ☐ Hold the B Button on the driver controller. The robot should rotate toward the tag and drive to your DESIRED_RANGE_METERS.
- ☐ Measure the actual stopping distance. If wrong, recheck CAMERA_HEIGHT_METERS and TARGET_HEIGHT_METERS first.
- ☐ Release B at any time to return to manual driving.

STEP 6: PID Tuning

After confirming the robot can align, tune the PID constants if movement feels wrong. Change one value at a time and redeploy after each change.

Constant	Symptom	Fix
kP_ROTATION	Robot turns too slowly toward the tag	Increase (e.g., 0.04 to 0.06)
kP_ROTATION	Robot shakes, oscillates, or overshoots	Decrease (e.g., 0.04 to 0.02)
kP_ROTATION	Robot turns away from the tag	Add a negative sign (e.g., -0.04)
kP_DISTANCE	Robot moves too slowly toward the goal	Increase the value
kP_DISTANCE	Robot slams into goal or overshoots distance	Decrease the value

NOTE: Both kP_ROTATION and kP_DISTANCE are in VisionSubsystem.java. Change the value, save, and redeploy. You do not need to change RobotContainer.java for tuning.

Troubleshooting

Symptom	Likely Cause	Fix
LimelightHelpers cannot be resolved	Vendor library not installed	Repeat Step 2. Check build.gradle for limelight entry.
VisionSubsystem cannot be resolved	File not created or wrong package	Confirm file is at src/main/java/frc/robot/subsystems/VisionSubsystem.java
B button does nothing	whileTrue binding not added	Confirm the NEW section in configureBindings() is present in RobotContainer.java
Robot turns away from tag	kP_ROTATION wrong sign	Change kP_ROTATION to -0.04 in VisionSubsystem.java
Robot stops at wrong distance	Wrong height measurements	Re-measure CAMERA_HEIGHT_METERS and TARGET_HEIGHT_METERS
getTV always false	Limelight name mismatch	Confirm LIMELIGHT_NAME in code matches name in Limelight dashboard Settings tab
tv/tx/ty missing in NetworkTables	Pipeline index not 0	In Limelight dashboard, confirm pipeline is set to index 0
Build fails after adding code	Missing RunCommand import	Confirm 'import edu.wpi.first.wpilibj2.command.RunCommand;' is in RobotContainer.java

Pre-Match Checklist

Item	Required Value / Action	Confirmed
Limelight dashboard loads	http://10.13.51.11:5801	☐
Pipeline type	AprilTag, tag36h11, index 0	☐
Stream settings	320x240, 15 FPS	☐
Tag detection in dashboard	Green box appears on tag	☐
Vendor library in build.gradle	limelight entry present	☐
VisionSubsystem.java created	Correct path and package	☐
RobotContainer.java updated	3 new lines added, nothing removed	☐
CAMERA_HEIGHT_METERS	Measured and updated	☐
TARGET_HEIGHT_METERS	Set from 2026 Game Manual	☐
CAMERA_PITCH_RADIANS	Measured and updated	☐
DESIRED_RANGE_METERS	Measured and updated	☐
Build and deploy successful	No errors in terminal	☐

Item	Required Value / Action	Confirmed
B Button tested on field	Robot aligns and stops at correct distance	[]
PID tuned	No oscillation or overshoot	[]